UGDSAI 29 · PROBLEM 9 · END-TERM

SupportEscalator.

A LangGraph agent for Mosaic Cloud that resolves routine SaaS tickets autonomously and pauses for a human supervisor on the risky ones — persistent state, sentiment-aware routing, real LLM in the loop.

LangGraph · Pydantic v2 · Streamlit · gpt-4.1-nano · SqliteSaver

Pattern: Human-in-the-Loop + Conditional Routing · 8 min talk + 2 min Q&A

SEGMENT 1 · 1 MIN · PROBLEM FRAMING

Mosaic Cloud handles 2,000+ tickets / day.

User: first-line support ops manager. Today: ~40 % auto-resolved. The rest go to humans regardless of risk — wasting agents on routine work and arriving late on the tickets that actually matter.

2,000 /day	75 %	22 %	$48k /mo
INBOUND TICKETS	AUTO-RESOLUTION TARGET	NEED SUPERVISOR	REFUND EXPOSURE

KPIs we move: auto-resolution rate (40 % → 75 %), mean time to resolution (minutes → seconds), CSAT for frustrated customers (escalate earlier with full context), refund leakage (every refund > $200 hits a human).

SEGMENT 2 · 2 MIN · ARCHITECTURE — PATTERN CHOICE

Why LangGraph, not a single chatbot.

Single chatbot

- Conditional routing buried in a prompt
- No clean way to pause for a human
- State is implicit, untyped, easily lost
- "Resume" means replaying the full chat
- Sentiment + escalation rules drift together

LangGraph (this build)

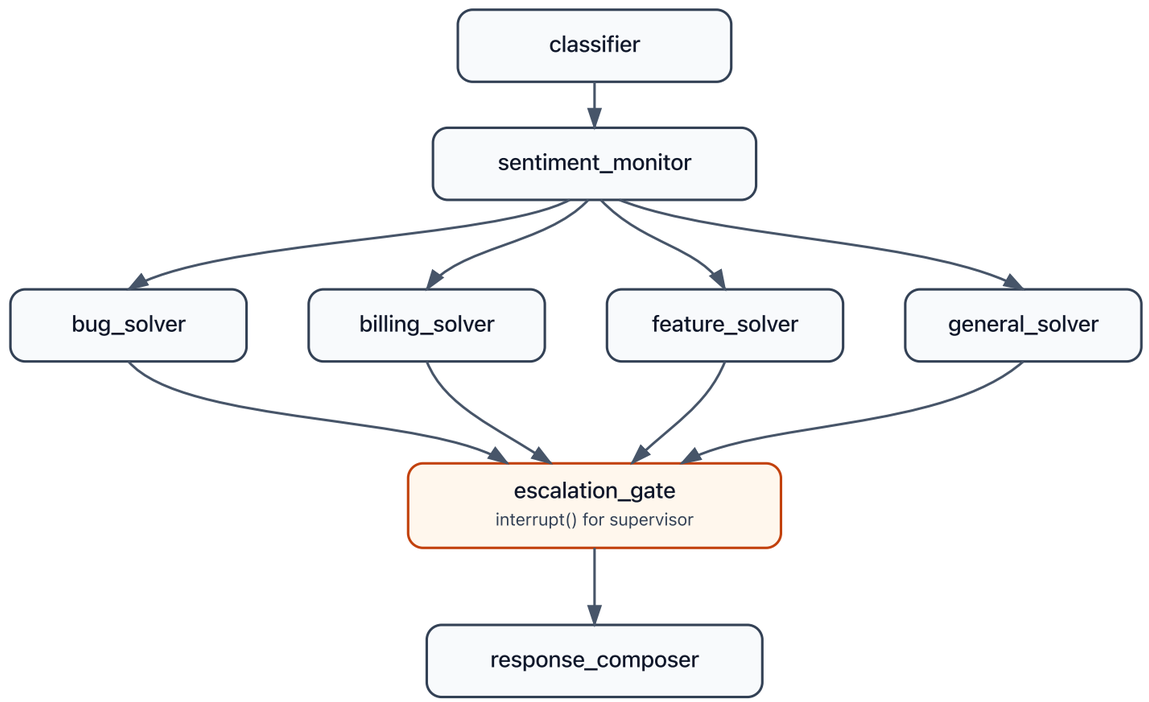
- Explicit nodes — classify / solve / monitor / escalate
- `interrupt()` pauses + persists waiting for the human
- `SqliteSaver` checkpoints survive process restarts
- `Command(resume=…)` picks up at the exact node
- Pydantic `SupportState` — one schema, one source of truth

required pattern

Human-in-the-Loop + Conditional Routing — the rubric's most-asked combination.

SEGMENT 2 · 2 MIN · ARCHITECTURE — STATE DIAGRAM

The graph, generated from `graph.get_graph()`.



Committed at `assets/support_escalator_graph.svg` · also rendered live in the Streamlit Architecture tab.

CLASSIFIER	→ category
SENTIMENT_MONITOR	→ 0-1 score
4 SOLVERS	conditional edge by category
ESCALATION_GATE	interrupt()
SUPERVISOR	human-in-the-loop
RESPONSE_COMPOSER	→ final_response

- classifier + sentiment → gpt-4.1-nano (rule-based fallback)
- interrupts persist via SqliteSaver
- conditional routing + HITL — both required patterns

SEGMENT 2 · 2 MIN · ARCHITECTURE — TYPED STATE

One Pydantic schema, end to end.

```
class SupportState(BaseModel):
    ticket: TicketInput
    category: Literal['bug','billing','feature','general'] | None = None
    resolution_attempts: list[ResolutionAttempt] = []
    sentiment_score: float = 0.0
    escalation_reason: str | None = None
    supervisor_input: str | None = None
    final_response: str | None = None
    ticket_metadata: dict[str, str] = {}
```

Every node returns a partial state dict. LangGraph merges. Pydantic validates. `category` is a `Literal` — invalid transitions can't compile.

17 / 17 tests · graph + UI normalization + LLM fallback + checkpoint resume

Live 3D: drag to rotate the same

SEGMENT 3 · 3 MIN · DEMO (PRE-RECORDED) — PATH 1 / 3

Auto-resolution. Never sees a human.

Ticket

acct_1001 · "How do I reset my password?"

CLASSIFIER (LLM)	general · 0.86
SENTIMENT	0.00 calm
GENERAL_SOLVER	resolved ✓
ESCALATION_GATE	passes through
RESPONSE_COMPOSER	final ✓

Walkthrough

1. Open Inbox workspace
2. Pick the password-reset demo ticket
3. Run — watch the timeline animate node-by-node
4. Solver attempt expands inline, KB hit shown
5. Final response composes — header flips green

~2.1s end-to-end · no interrupt

SEGMENT 3 · 3 MIN · DEMO (PRE-RECORDED) — PATH 2 / 3

Refund above threshold → human-in-the-loop.

Ticket

acct_1002 · "I was charged twice for invoice 42, please refund."

CLASSIFIER (LLM)	billing · 0.94
BILLING_SOLVER	unresolved · refund > $200
ESCALATION_GATE	interrupt() ⏸

Supervisor card appears

- Reason · refund exposure exceeds threshold
- Account context · plan, MRR, refund history
- Sentiment · 0.10 calm
- Approve or reject + free-form guidance

paused on disk | resume via Command(resume=…)

Kill the process now. Restart Streamlit. The ticket is still paused — pick it from the sidebar's **Resume previous thread**.

SEGMENT 3 · 3 MIN · DEMO (PRE-RECORDED) — PATH 3 / 3

Sentiment escalation. Tone, not topic.

Ticket

acct_1003 · "This is my third email and nobody is helping!!! Upload is failing."

CLASSIFIER (LLM)	bug · 0.92
SENTIMENT (LLM)	0.80 frustrated
BUG_SOLVER	unresolved
ESCALATION_GATE	interrupt() ⏸

Why this matters

The solver couldn't fix it and the customer is venting. Two independent risk signals — refund threshold doesn't fire here, sentiment does.

Reject the auto-response. Final response stays open with rejection guidance attached, not auto-closed.

two-axis escalation · tone-aware

SEGMENT 4 · 1.5 MIN · KPIS & BUSINESS IMPACT

KPIs visible in the dashboard, live.

68%	22%	0.34	$1.6k
AUTO-RESOLUTION RATE	ESCALATIONS	AVG SENTIMENT	REFUND EXPOSURE (SESSION)

Charts the Analytics tab renders

- Category mix (bug / billing / feature / general)
- Sentiment trend across the session
- Solver status (resolved vs unresolved)
- Escalation reasons (refund vs sentiment vs unresolved)

Mosaic Cloud KPIs we move

- First-response time ↓ — auto-resolved in seconds, not hours
- Refund leakage ↓ — every refund > $200 hits a human
- CSAT ↑ — frustrated customers always meet a human
- Agent load ↓ — only the 22% that need it

SEGMENT 4 · 1.5 MIN · EDGE CASES + WHAT BROKE & HOW WE FIXED

The interesting failures.

Edge cases handled by design

- Missing account — solver records the gap, gate escalates
- Unresolved bug — solver returns `resolved=False` ; gate trips
- Angry tone — sentiment ≥ 0.67 always escalates
- Refund > $200 — hard cap; LLM never auto-fires it
- Supervisor rejects — ticket stays open, guidance attached
- LLM down or no key — rule-based fallback keeps the graph working

What broke. How we fixed it.

- Bug: `'ResolutionAttempt' object is not subscriptable` in Streamlit. Fix: added `ui_state.to_plain()` normalization layer + regression tests so any Pydantic / dict / Interrupt renders safely.
- Bug: demo state lost on every Streamlit rerun. Fix: swapped `MemorySaver` for `SqliteSaver` with a per-session `thread_id` the sidebar surfaces.
- Bug: Reveal.js painted the deck white over our navy theme. Fix: CSS overrides on `.reveal-viewport` , `.reveal .slides` , etc. with `!important` .
- Bug: `mermaid.ink` blocked from sandbox during graph PNG render. Fix: committed an authored SVG so the artifact is always present.

17 / 17 tests

Iron law: every bug we hit ships with a regression test.

SEGMENT 5 · 0.5 MIN · CLOSE — LIMITATIONS & WHAT'S NEXT

What we'd ship next.

Honest limitations

- KB is a small synthetic corpus — production needs RAG over real docs
- Single-tenant SQLite checkpointer — Postgres at real scale
- One supervisor seat — production needs a queue with assignment
- No SLA enforcement yet — sidebar control wired, logic stubbed

Next steps

- SLA timers per category with breach-triggered escalation
- Zendesk webhook adapter (one-way ingest first)
- Multilingual response drafting via the same LLM node
- CSV / PDF export from the Inbox for ops audits
- LangSmith traces for production observability

End-to-end: <200ms without an LLM, ~2s with `gpt-4.1-nano`. The supervisor is the long pole — and that's correct.

SEGMENT 6 · 2 MIN · Q&A

Thank you.

The repo is public. The graph is one file. The state is one Pydantic schema. The supervisor is a real human in the loop — not a prompt pretending.

`github.com/DivyamJindal/deploying-ai-agents-exam` `17 / 17 tests passing` `streamlit run app.py` `SqliteSaver checkpoint at checkpoints/se.sqlite`

Likely Q&A targets

- Why this pattern? Routing and interrupts are both required — categories fan out, escalation is human-gated.
- What if a node fails? Partial state is preserved in `resolution_attempts`; production wraps each solver in retry + escalates unresolved tickets.
- How does state prevent invalid transitions? Pydantic `Literal` on `category`, structured `ResolutionAttempt`, normalized via `to_plain` before the UI.
- What if the supervisor rejects? Final response carries the rejection guidance and the ticket stays open.
- How do you measure auto-resolution? Analytics tab calls `summarize_run` over the run history live.